

CHAPTER 3

Unified Mathematical Framework for Disturbance-Aware Zero-Sum Pursuit–Evasion Control

3.1. Role and Boundaries of the Unified Framework

The previous chapter established the literature that supports zero-sum dynamic games, model-based Riccati methods, model-free quadratic Q-learning, persistent input estimation, and affine compensation. The purpose of the present chapter is different. Rather than continuing the literature review, this chapter defines the common mathematical problem that is used by all four algorithms in the thesis. More specifically, it establishes the state equation, the minimizing and maximizing inputs, the quadratic game objective, the saddle-point policies, the game algebraic Riccati equation (GARE), the quadratic action-value function, the persistent command-offset model, and the numerical conditions that are required before a computed solution can be interpreted as a valid zero-sum controller.

This common formulation is necessary because the model-based and model-free methods are not intended to solve different control problems. Model-based policy iteration, model-based value iteration, model-free Q-learning policy iteration, and model-free Q-learning value iteration all refer to the same discrete-time two-player game. Their principal distinction is the information that is used during the computation. The model-based methods evaluate matrix equations by using the known system matrices. The model-free methods estimate the corresponding quadratic relationships from sampled state and input data. Therefore, a meaningful comparison requires the plant, cost, player interpretation, command offsets, compensation cases, and validation metrics to remain fixed across the four methods [1, 2].

The chapter also clarifies what the current pursuit–evasion interpretation does and does not claim. The present MATLAB benchmark is a reduced-order linear zero-sum system inherited from the source algorithm files. It is interpreted as a local pursuit–evasion engagement in which the minimizing player acts as the pursuer and the maximizing player acts as the evader. However, the three-state benchmark is not presented as a complete nonlinear vehicle model containing planar position, heading angle, wheelbase, acceleration, and steering. Those physical states remain part of the future vehicle extension. In the current thesis, pursuit–evasion supplies the game interpretation,

while the controlled linear system supplies the mathematically traceable benchmark on which the disturbance-aware algorithm extensions are developed.

This separation is important for the structure of the thesis. Chapter 4 develops the two model-based methods from the common equations introduced here. Chapter 5 develops the two model-free methods by using the same state, cost, and player definitions. Chapter 6 then compares nominal operation, uncompensated command bias, estimated compensation, and exact compensation under a common MATLAB protocol. As a result, the present chapter acts as the point where the theoretical, algorithmic, and experimental parts of the thesis are joined.

The boundaries of this chapter are equally important. The chapter states the nominal mathematical targets and the common disturbance-aware execution model, but it does not reproduce the complete iteration-by-iteration procedures for all four algorithms. The detailed policy-evaluation, policy-improvement, value-update, Bellman-regression, and gain-recovery steps are reserved for Chapters 4 and 5. This organization prevents the algorithm chapters from repeating material while preserving one shared mathematical foundation for the later comparison.

3.2. Common Discrete-Time Zero-Sum Dynamics

The nominal system is a discrete-time linear plant with one minimizing input channel and one maximizing input channel. It is represented by

$$x_{k+1} = Ax_k + B_P u_{P,k} + B_E u_{E,k}, \quad (3.1)$$

where $x_k \in \mathbb{R}^n$ denotes the game state at time step k , $u_{P,k} \in \mathbb{R}^{m_P}$ denotes the pursuer input, and $u_{E,k} \in \mathbb{R}^{m_E}$ denotes the evader input. The matrices $A \in \mathbb{R}^{n \times n}$, $B_P \in \mathbb{R}^{n \times m_P}$, and $B_E \in \mathbb{R}^{n \times m_E}$ define the nominal state and player-input channels. In the MATLAB source files, B_P is denoted by B , while B_E is denoted by E . The notation in this chapter is used to make the pursuer and evader roles explicit.

The state equation in (3.1) is shared by both players. This means that the pursuer and evader do not operate separate plants. Rather, their inputs act on the same state, and the resulting state transition reflects the combined effect of both policies. The pursuer attempts to regulate the state in the direction favored by the performance objective, while the evader attempts to oppose that regulation. This coupled structure is what distinguishes the problem from two independent optimal-control problems [3, 4].

A state-space formulation is useful for three reasons. First, it makes the effect of each player explicit through B_P and B_E . Second, it supports a quadratic value function and a GARE-based saddle

solution. Third, it allows the same transition data to be reorganized into a quadratic Q-learning regression. The common state-space model therefore provides the analytical baseline for the model-based methods and the data-generating environment for the model-free methods.

The present benchmark is time invariant, fully observed, and deterministic during the nominal control calculation. These assumptions are intentionally restrictive. Time variation, partial state measurement, stochastic process noise, actuator saturation, and nonlinear dynamics are important extensions, but including them at this stage would make it difficult to determine whether an observed difference is caused by the original algorithm, the command-offset extension, or an unrelated modeling change. Consequently, the current formulation begins with the smallest system that is able to preserve the zero-sum saddle structure and the distinction between known-model and data-based computation.

3.3. Pursuer, Evader, and Common Notation

The minimizing and maximizing players are interpreted as a pursuer and an evader. Under this interpretation, the pursuer attempts to reduce a selected relative-state error, while the evader attempts to preserve or enlarge that error. The interpretation follows the standard zero-sum pursuit–evasion structure in which both players affect the same engagement state but act with opposing objectives [5, 3].

For the current reduced-order system, an auxiliary engagement output is written as

$$y_k = Cx_k. \quad (3.2)$$

The verified MATLAB benchmark uses

$$C = \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}. \quad (3.3)$$

Thus, $y_k = x_{1,k} - x_{3,k}$ can be interpreted as a reduced relative engagement variable. The output is useful for connecting the state-space benchmark to pursuit–evasion language, but it does not define the state-penalty matrix in the present implementation. The numerical studies use $Q = I_3$, so all three states are penalized directly in the quadratic game objective.

This interpretation is intentionally more limited than a geometric vehicle capture condition. In a complete planar pursuit–evasion model, capture may be defined through the Euclidean separation between two position vectors. In the current linear benchmark, the output in (3.2) is a local

engagement proxy that supports interpretation and reporting. Therefore, the chapter does not claim that $y_k = 0$ is identical to physical contact between two vehicles. Instead, it is a reduced-order quantity that allows the four algorithms and the command-offset extension to be studied without changing the original benchmark matrices.

The pursuer and evader policies are both calculated from the same game state. Thus, the system uses a feedback information pattern rather than open-loop input sequences. At each time step, the current state is supplied to both policies, each player generates a command, and the two realized inputs jointly determine the next state. This feedback structure is central to the later policy-iteration and Q-learning derivations because both methods seek stationary gain matrices rather than one-time input trajectories.

Table 3.1 summarizes the notation used throughout Chapters 3–6.

Table 3.1: Common symbols used in the disturbance-aware zero-sum formulation.

Symbol	Meaning
x_k	Shared game state at time step k .
$u_{P,k}$	Pursuer or minimizing-player input.
$u_{E,k}$	Evader or maximizing-player input.
A	Nominal discrete-time state matrix.
B_P, B_E	Pursuer and evader input matrices.
C	Auxiliary engagement-output matrix.
Q	Positive semidefinite state-penalty matrix.
R_P, R_E	Positive definite pursuer and evader input-weight matrices.
P	Quadratic state-value matrix.
H	Quadratic action-value matrix.
L, K	Pursuer and evader state-feedback gain matrices.
b_P, b_E	Persistent realized-input offsets in the two command channels.
$\widehat{b}_P, \widehat{b}_E$	Estimated offsets used by the affine compensation layer.
$\widetilde{b}_P, \widetilde{b}_E$	Residual offset errors after compensation.

3.4. Quadratic Game Objective and Saddle-Point Policies

The infinite-horizon performance index is represented by

$$J(x_0, u_P, u_E) = \sum_{k=0}^{\infty} \left(x_k^T Q x_k + u_{P,k}^T R_P u_{P,k} - u_{E,k}^T R_E u_{E,k} \right), \quad (3.4)$$

where $Q \succeq 0$, $R_P \succ 0$, and $R_E \succ 0$. The pursuer minimizes J , while the evader maximizes the same quantity. The saddle objective can therefore be written conceptually as

$$J^*(x_0) = \min_{u_P} \max_{u_E} J(x_0, u_P, u_E). \quad (3.5)$$

The sign structure in (3.4) is essential. The pursuer is penalized for state deviation and its own control effort. The evader appears with a negative input penalty because it acts as the maximizing player. This does not mean that arbitrarily large evader inputs are allowed. Rather, R_E and the saddle-point curvature conditions limit the maximizing direction and determine whether the stage game is well posed. In other words, the negative sign creates the adversarial objective, while the matrix conditions introduced later prevent the maximization from becoming unbounded.

For the verified MATLAB benchmark, the weights are

$$Q = I_3, \quad R_P = 1, \quad R_E = \gamma^2, \quad \gamma = 1. \quad (3.6)$$

The parameter γ is consistent with the H_∞ interpretation of the source zero-sum game, but the present thesis uses pursuer–evader language so that the same minimizing and maximizing channels can be interpreted within a pursuit–evasion setting [1, 6].

The objective in (3.4) is evaluated using the inputs that actually enter the plant. This distinction becomes important after command bias is introduced. A controller may compute a nominal command, but the cost accumulated by the physical or simulated plant depends on the realized input in the state equation. Consequently, the later comparisons use the same cost definition for the nominal, biased, and compensated trajectories while changing only the relationship between commanded and realized inputs.

3.4.1. Quadratic Value Function and Feedback Saddle Policies

For a stabilizing linear-quadratic zero-sum game, the value function is assumed to have the quadratic form

$$V(x_k) = x_k^\top P x_k, \quad (3.7)$$

where $P = P^\top \succeq 0$ is the game value matrix. The feedback policies are represented by

$$u_{P,k} = L x_k, \quad u_{E,k} = K x_k, \quad (3.8)$$

where $L \in \mathbb{R}^{m_P \times n}$ and $K \in \mathbb{R}^{m_E \times n}$. The sign convention in (3.8) is retained from the MATLAB files. Any negative feedback direction is therefore contained in the numerical values of L and K rather than written as an additional minus sign outside the gains.

Substituting (3.8) into (3.1) gives the nominal closed-loop system

$$x_{k+1} = A_{cl}x_k, \quad A_{cl} = A + B_P L + B_E K. \quad (3.9)$$

A stabilizing saddle solution requires A_{cl} to be Schur stable, meaning that every eigenvalue lies strictly inside the unit circle. This condition is written as

$$\rho(A_{cl}) < 1, \quad (3.10)$$

where $\rho(\cdot)$ denotes the spectral radius.

The saddle policies must also satisfy the minimax ordering. For every admissible state, the pursuer policy should minimize the value against the evader policy, while the evader policy should maximize the value against the pursuer policy. Conceptually, this is expressed by

$$J(Lx, K'x) \leq J(Lx, Kx) \leq J(L'x, Kx) \quad (3.11)$$

for admissible alternative gains L' and K' . The exact inequality depends on the policy class and initial state, but the expression captures the central interpretation: neither player can improve its objective through a unilateral admissible change when the other player holds the saddle policy fixed [3, 4].

The value matrix P is not only a cost summary. It also connects the model-based and model-free portions of the thesis. In the model-based methods, P is evaluated or updated directly from the system matrices. In the model-free methods, the action-value matrix H is estimated from data, after which the policy gains and an equivalent state-value relationship are recovered. Thus, P and H represent two views of the same nominal game.

3.5. Equivalent Nominal Value Representations

The four algorithms considered in this thesis seek the same nominal saddle solution through two equivalent quadratic representations. The model-based algorithms operate directly on the state-value matrix P , while the model-free algorithms estimate a joint state-action matrix H . This section

defines those common mathematical objects without reproducing the complete iteration procedures that are developed in Chapters 4 and 5.

3.5.1. Generalized Algebraic Riccati Game Target

The GARE provides the steady-state model-based solution. To state the equation compactly, define the block game matrix

$$\mathcal{G}(P) = \begin{bmatrix} R_P + B_P^\top P B_P & B_P^\top P B_E \\ B_E^\top P B_P & B_E^\top P B_E - R_E \end{bmatrix}. \quad (3.12)$$

The stacked gain matrix is

$$\begin{bmatrix} L \\ K \end{bmatrix} = -\mathcal{G}(P)^{-1} \begin{bmatrix} B_P^\top P A \\ B_E^\top P A \end{bmatrix}. \quad (3.13)$$

In numerical implementation, (3.13) is evaluated through a linear solve rather than by explicitly forming the inverse.

The value matrix satisfies

$$\begin{aligned} P &= Q + A^\top P A \\ &\quad - \begin{bmatrix} A^\top P B_P & A^\top P B_E \end{bmatrix} \mathcal{G}(P)^{-1} \begin{bmatrix} B_P^\top P A \\ B_E^\top P A \end{bmatrix}. \end{aligned} \quad (3.14)$$

Equation (3.14) is the common nominal target for the two model-based algorithms and the corresponding model-free Q-learning procedures [4, 7].

The block matrix in (3.12) must be numerically invertible. However, invertibility alone is not sufficient. Define

$$H_{PP} = R_P + B_P^\top P B_P, \quad (3.15)$$

$$H_{PE} = B_P^\top P B_E, \quad H_{EP} = B_E^\top P B_P, \quad (3.16)$$

$$H_{EE} = B_E^\top P B_E - R_E. \quad (3.17)$$

The minimizing-player block must be positive definite,

$$H_{PP} \succ 0, \quad (3.18)$$

and the maximizing-player Schur complement must be negative definite,

$$S_E = H_{EE} - H_{EP}H_{PP}^{-1}H_{PE} \prec 0. \quad (3.19)$$

The first condition ensures local convexity in the minimizing-player input. The second ensures local concavity in the maximizing-player input after the minimizing direction is accounted for. These conditions distinguish a valid saddle point from a stationary point that has the wrong curvature.

An equivalent closed-loop policy-evaluation identity is

$$P = Q + L^\top R_P L - K^\top R_E K + A_{\text{cl}}^\top P A_{\text{cl}}. \quad (3.20)$$

This form is useful for verification because a converged P , L , and K should satisfy both the GARE form and the closed-loop Lyapunov form within numerical tolerance. The detailed use of (3.20) in policy iteration, and the repeated Riccati mapping used in value iteration, are presented in Chapter 4.

3.5.2. Quadratic Action-Value Representation

The model-free methods use a quadratic action-value function. Define the joint state-action vector

$$z_k = \begin{bmatrix} x_k \\ u_{P,k} \\ u_{E,k} \end{bmatrix} \in \mathbb{R}^{n_z}, \quad n_z = n + m_P + m_E. \quad (3.21)$$

The one-step action-value function is

$$Q(z_k) = z_k^\top H z_k, \quad (3.22)$$

where $H = H^\top$ is partitioned as

$$H = \begin{bmatrix} H_{xx} & H_{xP} & H_{xE} \\ H_{Px} & H_{PP} & H_{PE} \\ H_{Ex} & H_{EP} & H_{EE} \end{bmatrix}. \quad (3.23)$$

For a known value matrix P , the exact nominal action-value matrix is

$$H = \begin{bmatrix} Q + A^\top P A & A^\top P B_P & A^\top P B_E \\ B_P^\top P A & R_P + B_P^\top P B_P & B_P^\top P B_E \\ B_E^\top P A & B_E^\top P B_P & B_E^\top P B_E - R_E \end{bmatrix}. \quad (3.24)$$

The model-free algorithms do not construct (3.24) by substituting the system matrices into the formula. Instead, they estimate the unique elements of H from sampled Bellman relationships [2, 7].

The saddle policies are recovered directly from the input-related blocks of H :

$$\begin{bmatrix} L \\ K \end{bmatrix} = - \begin{bmatrix} H_{PP} & H_{PE} \\ H_{EP} & H_{EE} \end{bmatrix}^{-1} \begin{bmatrix} H_{Px} \\ H_{Ex} \end{bmatrix}. \quad (3.25)$$

Equation (3.25) has the same block saddle structure as the model-based gain equation. This is the central reason that quadratic Q-learning is a direct model-free counterpart to the GARE solution rather than an unrelated reinforcement-learning controller.

Because H is symmetric, it contains

$$n_H = \frac{n_z(n_z + 1)}{2} \quad (3.26)$$

unique coefficients. A symmetric quadratic feature vector $\phi(z_k)$ is therefore constructed from squared terms and doubled cross terms. The sampled Bellman equations take the regression form

$$\Phi\theta \approx r, \quad (3.27)$$

where θ contains the unique coefficients of H , Φ is the feature matrix, and r is the target vector formed from stage costs and successor-state terms. At minimum,

$$\text{rank}(\Phi) = n_H \quad (3.28)$$

should hold for exact least-squares identification of the quadratic parameters. Full numerical rank is necessary, but the condition number also matters. A feature matrix can be technically full rank while remaining so ill-conditioned that the estimated coefficients are dominated by numerical error. Consequently, the model-free chapters report both rank and conditioning rather than treating the

least-squares solution as valid merely because MATLAB returns a vector.

Model-free policy iteration and model-free value iteration differ in how the Bellman target is formed and how the policy is updated. Policy iteration evaluates the Q-function associated with a current gain pair and then improves the pair through the saddle-policy recovery equation. Value iteration updates the value or action-value approximation directly from the sampled optimality relation. The detailed update equations are presented in Chapter 5 so that this chapter can remain focused on the mathematical objects shared by both methods. The important point is that neither method uses A , B_P , or B_E directly in its Q-learning regression.

Table 3.2 summarizes the information structures and computational roles of the four methods without duplicating the algorithmic derivations.

Table 3.2: Common roles and information structures of the four thesis algorithms.

Method	Information used in nominal update	Principal computational structure
Model-based policy iteration	Known A , B_P , and B_E	Lyapunov policy evaluation followed by coupled saddle-gain improvement.
Model-based value iteration	Known A , B_P , and B_E	Repeated GARE or Bellman value mapping followed by gain recovery.
Model-free Q-learning policy iteration	Sampled states, realized inputs, costs, and successor states	Quadratic Q-function regression for fixed policies followed by policy improvement.
Model-free Q-learning value iteration	Sampled states, realized inputs, costs, and successor states	Sample-based value or Q-function updates followed by gain recovery.

3.6. Persistent Command-Channel Offset Model

The nominal algorithms assume that the command generated by each policy is the input that enters the plant. The disturbance-aware extension removes this idealized equivalence by separating the commanded input from the realized input. This distinction is important because a policy can be mathematically correct for the nominal game while the executed plant still experiences a persistent offset. The offset does not require the GARE or the quadratic Q-function to be replaced. Rather, it introduces an affine execution-side term that must be identified and compensated after the nominal saddle policies have been obtained.

For the pursuer, the realized input is represented as

$$u_{P,k}^{\text{act}} = u_{P,k}^{\text{cmd}} + b_P, \quad (3.29)$$

and for the evader,

$$u_{E,k}^{\text{act}} = u_{E,k}^{\text{cmd}} + b_E, \quad (3.30)$$

where $b_P \in \mathbb{R}^{m_P}$ and $b_E \in \mathbb{R}^{m_E}$ are constant or slowly varying command-channel offsets over the calibration and simulation windows. The offsets are matched disturbances because each term enters through the same input matrix as the corresponding player command. The realized plant is therefore

$$x_{k+1} = Ax_k + B_P u_{P,k}^{\text{act}} + B_E u_{E,k}^{\text{act}}. \quad (3.31)$$

If the nominal feedback commands are applied without compensation,

$$u_{P,k}^{\text{cmd}} = Lx_k, \quad u_{E,k}^{\text{cmd}} = Kx_k, \quad (3.32)$$

then the closed-loop dynamics become

$$x_{k+1} = A_{\text{cl}}x_k + B_P b_P + B_E b_E. \quad (3.33)$$

The constant affine forcing term is

$$d_b = B_P b_P + B_E b_E. \quad (3.34)$$

When $I - A_{\text{cl}}$ is nonsingular, the corresponding biased equilibrium is

$$x_{\infty}^{\text{bias}} = (I - A_{\text{cl}})^{-1} d_b. \quad (3.35)$$

Equation (3.35) distinguishes stability from regulation. A Schur-stable feedback matrix can still drive the realized plant toward a nonzero equilibrium when the command offsets remain uncompensated. Thus, the disturbance-aware problem is not created by a failure of the nominal Riccati or Q-learning solution. It is created by the difference between the policy command and the input that is actually applied. This observation supplies the mathematical reason for adding an estimation and compensation layer while retaining the nominal gain pair.

The same offset model can represent several execution-side effects. A repeatable acceleration

error, steering-center offset, motor-command conversion error, actuator calibration mismatch, or persistent input-reconstruction error can be approximated by b_P and b_E over a finite experiment. The thesis does not claim that every fault is constant. Instead, the constant-offset assumption provides a transparent first disturbance model whose influence can be derived exactly and whose compensation can be verified without changing the four source algorithms [8, 9].

3.7. Offset Estimation under Model-Based and Model-Free Information

The model-based and model-free methods require bias estimates for the same purpose, but they do not possess the same information. The model-based algorithms know A , B_P , and B_E , whereas the model-free Bellman regressions are intentionally constructed without those matrices. Consequently, the estimation procedures are matched in objective but are not forced into the same numerical form.

3.7.1. Model-Based Transition-Residual Estimation

For the model-based case, define the one-step transition residual

$$r_k = x_{k+1} - Ax_k - B_P u_{P,k}^{\text{cmd}} - B_E u_{E,k}^{\text{cmd}}. \quad (3.36)$$

Substituting the realized-input model gives

$$r_k = \underbrace{\begin{bmatrix} B_P & B_E \end{bmatrix}}_{G_b} \underbrace{\begin{bmatrix} b_P \\ b_E \end{bmatrix}}_b + v_k, \quad (3.37)$$

where v_k collects transition-measurement noise and unmodeled effects. For a constant bias over N_{cal} samples, the sample-mean residual satisfies

$$\bar{r} = G_b \bar{b} + \bar{v}. \quad (3.38)$$

The least-squares estimate is

$$\hat{\bar{b}} = G_b^\dagger \bar{r}, \quad (3.39)$$

where $(\cdot)^\dagger$ denotes the Moore–Penrose pseudoinverse. In the MATLAB implementation, the estimate is obtained through a linear least-squares solve rather than by explicitly forming an inverse. This formulation uses the known plant matrices only in the model-based calibration stage and leaves

the original GARE policy or value update unchanged.

The two individual offsets are identifiable from the transition residual only when

$$\text{rank}(G_b) = m_P + m_E. \quad (3.40)$$

If the two input directions are linearly dependent, the net affine disturbance d_b may still be identifiable even when the separate player offsets are not. This rank condition is therefore a physical identifiability requirement rather than only a numerical convenience.

For independent isotropic transition noise with covariance $\sigma_v^2 I$, the covariance of the sample-mean least-squares estimate is

$$\text{Cov}(\hat{b}) = \frac{\sigma_v^2}{N_{\text{cal}}} (G_b^\top G_b)^{-1}, \quad (3.41)$$

provided that G_b has full column rank. This expression supplies the theoretical individual-estimate standard deviations used in the Monte Carlo validation of Chapter 6.

3.7.2. Model-Free Command-Mismatch Estimation

For the model-free case, the source implementation assumes that commanded and realized inputs can be measured during a calibration period. The offsets are estimated directly as

$$\hat{b}_P = \frac{1}{N_{\text{cal}}} \sum_{k=1}^{N_{\text{cal}}} \left(u_{P,k}^{\text{meas}} - u_{P,k}^{\text{cmd}} \right), \quad (3.42)$$

$$\hat{b}_E = \frac{1}{N_{\text{cal}}} \sum_{k=1}^{N_{\text{cal}}} \left(u_{E,k}^{\text{meas}} - u_{E,k}^{\text{cmd}} \right). \quad (3.43)$$

These equations do not use A , B_P , or B_E . Therefore, the Q-learning update remains model free with respect to the plant dynamics. The command-mismatch calculation is an execution-side measurement step rather than a hidden model-based reconstruction inside the Bellman regression.

Separate measured inputs are assumed in (3.42) and (3.43). If those measurements are unavailable, additional structure such as an observer, an augmented state, a plant model, or another identifiability assumption would be required [10]. Stating this limitation is important because the compensation results depend not only on the controller, but also on what is measured during calibration.

3.8. Affine Compensation and Residual Closed-Loop Behavior

After the offsets are estimated, the policy-generated inputs are modified before they enter the biased command channels. The compensated commands are

$$u_{P,k}^{\text{cmd}} = Lx_k - \widehat{b}_P, \quad (3.44)$$

$$u_{E,k}^{\text{cmd}} = Kx_k - \widehat{b}_E. \quad (3.45)$$

The realized inputs are therefore

$$u_{P,k}^{\text{act}} = Lx_k - \widehat{b}_P + b_P, \quad (3.46)$$

$$u_{E,k}^{\text{act}} = Kx_k - \widehat{b}_E + b_E. \quad (3.47)$$

Define the residual bias errors

$$\widetilde{b}_P = b_P - \widehat{b}_P, \quad \widetilde{b}_E = b_E - \widehat{b}_E. \quad (3.48)$$

Substitution into the plant gives

$$x_{k+1} = A_{\text{cl}}x_k + B_P\widetilde{b}_P + B_E\widetilde{b}_E. \quad (3.49)$$

Equation (3.49) contains the complete disturbance-aware interpretation. When the estimates are exact,

$$\widehat{b}_P = b_P, \quad \widehat{b}_E = b_E, \quad (3.50)$$

the residual offsets vanish and

$$x_{k+1} = A_{\text{cl}}x_k. \quad (3.51)$$

Thus, exact compensation recovers the nominal closed-loop dynamics without recomputing P , H , L , or K . The exact case is not presented as an implementable solution when the offsets are unknown. It is a verification case. If exact cancellation does not reproduce the nominal trajectory to numerical precision, the command and realized-input bookkeeping is inconsistent.

When the estimates contain error, the compensated plant remains subject to a smaller affine term. Provided that $I - A_{\text{cl}}$ is nonsingular, its residual equilibrium is

$$x_{\infty}^{\text{comp}} = (I - A_{\text{cl}})^{-1} (B_P\widetilde{b}_P + B_E\widetilde{b}_E). \quad (3.52)$$

This expression separates the three quantities that determine the final regulation error. The first is the estimation error itself. The second is the direction through which each residual enters the state. The third is the closed-loop amplification contained in $(I - A_{\text{cl}})^{-1}$. Therefore, a small scalar bias-estimation error does not automatically guarantee an equally small state error. A nearly marginal closed loop can amplify a constant residual more strongly than a well-damped closed loop.

The deviation from the nominal trajectory is defined as

$$d_k = x_k^{\text{case}} - x_k^{\text{nom}}. \quad (3.53)$$

For a matched initial condition, $d_0 = 0$, and the deviation recurrence is

$$d_{k+1} = A_{\text{cl}} d_k + B_P \tilde{b}_P + B_E \tilde{b}_E. \quad (3.54)$$

Its finite-step solution is

$$d_k = \sum_{j=0}^{k-1} A_{\text{cl}}^j (B_P \tilde{b}_P + B_E \tilde{b}_E). \quad (3.55)$$

The MATLAB implementations compare this analytical recurrence with the simulated trajectory difference. Agreement to floating-point precision verifies that the observed deviation is caused by the intended affine forcing rather than by inconsistent simulation logic.

The affine compensation layer preserves the nominal feedback gains. This design decision is central to the thesis claim. Each source algorithm computes L and K according to its original policy-iteration or value-iteration structure. Calibration estimates b_P and b_E , and execution subtracts those estimates before the commands enter the biased plant. The extension is therefore traceable to the original algorithms while remaining mathematically sufficient to remove a matched constant command offset.

3.9. Common Validation Cases

Each algorithm is evaluated through the same four execution cases. The cases are defined here so that Chapters 4–6 do not change the disturbance model or the meaning of compensation from one method to another.

The game matrices, cost weights, initial condition, true offsets, simulation horizon, and reporting definitions remain common across the four methods. Therefore, the comparison separates three questions. First, does the nominal algorithm converge to a valid saddle solution? Second, how

Table 3.3: Common execution cases used across the four algorithms.

Case	Definition and purpose
Nominal operation	The command offsets are zero and the feedback commands enter the plant directly. This case supplies the reference trajectory and finite-window cost.
Uncompensated off-sets	The true offsets are added to the policy commands, but no estimates are subtracted. This case isolates the effect of execution-side mismatch when the nominal gains are unchanged.
Estimated compensation	The offsets are estimated according to the information structure of the algorithm and are subtracted from the commanded inputs. This case measures practical recovery in the presence of calibration error.
Exact compensation	The true offsets are used in place of the estimates. This case verifies the algebra and implementation rather than representing a realizable controller with unknown biases.

strongly does that fixed gain pair respond to persistent command offsets? Third, how much of the nominal behavior is recovered after estimated or exact compensation?

The common protocol does not predetermine that one algorithm must outperform the others. Under matched assumptions and exact arithmetic, all four methods seek the same nominal saddle policies. Their practical differences are expected to appear in convergence behavior, model information, data requirements, conditioning, and sensitivity to calibration error. Therefore, the evaluation emphasizes common diagnostics rather than presenting model-based and model-free control as unrelated competitors.

3.10. Common Numerical Metrics and Validity Conditions

A zero-sum iteration can stop changing without producing a valid saddle solution. Likewise, a compensation routine can appear to reduce an error while still being inconsistent with the commanded and realized inputs. For these reasons, the MATLAB study uses two connected layers of verification. The first verifies the nominal saddle solution. The second applies that verified gain pair to the common disturbance-aware benchmark.

3.10.1. Verification of the Nominal Saddle Solution

The first check is closed-loop stability. After the gains are recovered, the nominal matrix in (3.9) must satisfy (3.10). For policy iteration, this condition is also checked during policy evaluation

because the infinite-horizon Lyapunov equation requires an admissible closed loop.

The second check is local saddle curvature. The symmetric part of the minimizing-player block must be positive definite, while the symmetric part of the maximizing-player Schur complement must be negative definite:

$$\lambda_{\min}\left(\frac{H_{PP} + H_{PP}^T}{2}\right) > 0, \quad (3.56)$$

$$\lambda_{\max}\left(\frac{S_E + S_E^T}{2}\right) < 0. \quad (3.57)$$

These tests distinguish a valid minimax saddle point from a stationary point having the wrong curvature. Symmetrization prevents small floating-point asymmetries from being treated as meaningful game structure.

The third check concerns numerical invertibility. The reciprocal condition number of the block game matrix should satisfy

$$\text{rcond}(\mathcal{G}) > \varepsilon_{\text{cond}}. \quad (3.58)$$

The MATLAB implementations do not explicitly compute $\mathcal{G}^{-1}$. Instead, the block gain equations are evaluated through linear solves using the backslash operator. This approach is more efficient and numerically more reliable than multiplying by an explicitly formed inverse [11].

The fourth check evaluates the defining Riccati equation. Let $\mathcal{T}(P)$ denote the right-hand side of (3.14). The residual is

$$R_{\text{GARE}} = P - \mathcal{T}(P), \quad (3.59)$$

and the normalized residual is

$$\varepsilon_{\text{GARE}} = \frac{\|R_{\text{GARE}}\|_F}{1 + \|P\|_F}. \quad (3.60)$$

This check is stronger than monitoring only the change between successive iterates. An update sequence can stagnate before reaching an accurate fixed point, whereas the residual directly tests the equation that the final value matrix is supposed to satisfy.

The policy-evaluation identity in (3.20) provides a second residual,

$$R_{\text{Lyap}} = P - \left(Q + L^T R_P L - K^T R_E K + A_{\text{cl}}^T P A_{\text{cl}}\right), \quad (3.61)$$

which is also normalized by $1 + \|P\|_F$. Agreement between the GARE and Lyapunov forms is particularly important for policy iteration.

The model-free methods require additional checks. The quadratic feature matrix must have full numerical rank, its condition number must remain acceptable, and the estimated H matrix must be symmetric to numerical tolerance. The sampled Bellman residual and the changes in L and K are also reported. When a model-based reference is used only for verification, relative gain errors may be computed as

$$\varepsilon_L = \frac{\|L_Q - L_{\text{ref}}\|_F}{1 + \|L_{\text{ref}}\|_F}, \quad \varepsilon_K = \frac{\|K_Q - K_{\text{ref}}\|_F}{1 + \|K_{\text{ref}}\|_F}. \quad (3.62)$$

These quantities are implementation diagnostics and are not supplied to the learner during Q-function regression.

3.10.2. Verification of Estimation and Compensation

The disturbance-aware layer is verified by reporting the true offsets, estimated offsets, signed and absolute estimation errors, theoretical and empirical estimator spread, and the residual bias terms. The exact-compensation trajectory error is defined as

$$\varepsilon_{\text{exact}} = \max_{0 \leq k \leq N_{\text{sim}}} \|x_k^{\text{exact}} - x_k^{\text{nom}}\|_2. \quad (3.63)$$

This quantity should be near floating-point precision when the exact and nominal cases use the same gain pair, initial condition, simulation horizon, and arithmetic.

The principal trajectory metric is the cost-weighted deviation

$$\|d_k\|_Q = \sqrt{d_k^\top Q d_k}. \quad (3.64)$$

Because $Q = I_3$ in the present benchmark, this metric is numerically equal to the Euclidean state-deviation norm. Writing the metric in the Q -weighted form keeps it aligned with the state penalty in the game objective and allows the same definition to remain valid if a different positive semidefinite Q is used in later work.

The finite-window accumulated game cost is reported as

$$J_N = \sum_{k=0}^{N_{\text{sim}}-1} \left[x_k^\top Q x_k + (u_{P,k}^{\text{act}})^\top R_P u_{P,k}^{\text{act}} - (u_{E,k}^{\text{act}})^\top R_E u_{E,k}^{\text{act}} \right]. \quad (3.65)$$

This quantity is a reproducible summary of the simulated interval and is not substituted for the infinite-horizon theoretical objective. Because the evader contribution enters with a negative sign, J_N is interpreted together with the state histories, realized inputs, convergence diagnostics, and

residual-bias measures rather than as a stand-alone measure of pursuit quality.

Table 3.4 summarizes the numerical checks used across the thesis.

Table 3.4: Numerical checks used across the four algorithms.

Check	Interpretation
Iteration change	Determines whether the value matrix, Q-function, and gain sequences have stopped changing.
GARE residual	Tests whether the converged model-based value matrix satisfies the defining Riccati equation.
Lyapunov residual	Tests whether the converged policy-iteration result satisfies the fixed-policy evaluation identity.
Closed-loop spectral radius	Verifies Schur stability of the nominal pursuer–evader gain pair.
Pursuer curvature	Verifies positive curvature in the minimizing-player input.
Evader Schur complement	Verifies negative curvature in the maximizing-player input after eliminating the pursuer block.
Reciprocal condition number	Detects near-singularity of the block game matrix before a gain solve is accepted.
Feature rank and conditioning	Determines whether the model-free quadratic parameters are identifiable and numerically reliable.
Bellman residual	Measures consistency of the estimated Q-function with the sampled Bellman equation.
Bias-estimation error	Measures calibration accuracy in each command channel.
Estimator spread	Compares empirical individual-estimate standard deviation with the theoretical covariance prediction.
Cost-weighted deviation	Measures departure from the nominal trajectory in the state metric used by the game.
Command-channel error	Measures the difference between the realized input and the policy input.
Analytical recurrence mismatch	Verifies agreement between the simulated state deviation and the affine deviation recurrence.

Check	Interpretation
Exact-compensation error	Verifies that exact bias cancellation reproduces the nominal closed-loop trajectory.

3.11. Common Numerical Benchmark

The common protocol is established before the numerical matrices are introduced. This ordering is intentional. The purpose of the benchmark is first to define what remains common across the algorithms and only then to state the particular numerical realization inherited from the source files.

The state matrix is

$$A = \begin{bmatrix} 0.906488 & 0.0816012 & -0.000500000 \\ 0.0741349 & 0.901210 & -0.000708383 \\ 0 & 0 & 0.132655 \end{bmatrix}, \quad (3.66)$$

and the pursuer and evader input matrices are

$$B_P = \begin{bmatrix} -0.00150808 \\ -0.00960000 \\ 0.867345 \end{bmatrix}, \quad B_E = \begin{bmatrix} 0.00951892 \\ 0.000383730 \\ 0 \end{bmatrix}. \quad (3.67)$$

The values in A define two relatively persistent and mutually coupled states together with a faster auxiliary state. The first two diagonal entries are near 0.9, so those components decay slowly in open loop and exchange information through the off-diagonal terms 0.0816012 and 0.0741349. The third state has a much smaller diagonal value, 0.132655, and therefore decays more rapidly when it is not being driven. The small negative entries in the third column connect that faster internal state back into the first two components. This organization produces a reduced-order dynamic chain rather than three independent scalar modes.

The player matrices create different control directions. The pursuer input has its dominant effect on the third state through the entry 0.867345, while its direct influence on the first two states is comparatively small. Consequently, the pursuer affects the full state through the internal dynamics of the plant. The evader input acts most strongly on the first state through 0.00951892, has a weaker

effect on the second state, and has no direct third-state component. Thus, the two players do not possess identical authority even though both use scalar inputs. The unequal input directions are part of what gives the benchmark a nontrivial zero-sum structure.

The auxiliary engagement output and the quadratic weights are

$$C = \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}, \quad Q = I_3, \quad R_P = 1, \quad R_E = 1. \quad (3.68)$$

The output $y_k = x_{1,k} - x_{3,k}$ provides a reduced pursuit–evasion interpretation, while $Q = I_3$ penalizes all three state components in the actual benchmark cost. This distinction is important because replacing Q with $C^\top C$ would define a different zero-sum game and would change the computed value matrix, feedback gains, convergence histories, and numerical results.

The numerical matrices are retained from the source implementation so that the thesis extension remains traceable to the original algorithms. They are not claimed to be a direct discretization of planar vehicle position, heading, acceleration, and steering. Instead, they define the reduced-order local game on which the disturbance-aware mathematics is verified. The pursuit–evasion interpretation is supplied by the minimizing and maximizing roles and by the auxiliary relative engagement output.

The common initial condition is

$$x_0 = \begin{bmatrix} 10 & 5 & -2 \end{bmatrix}^\top. \quad (3.69)$$

This choice excites all three states and prevents the numerical comparison from being reduced to a single-mode response. The persistent scalar offsets are

$$b_P = 0.040, \quad b_E = -0.010. \quad (3.70)$$

Their opposite signs allow the two command channels to produce competing affine effects. The values are large enough to create a visible difference from the nominal response while remaining small relative to the dynamic range of the simulated commands.

The common closed-loop horizon is $N_{\text{sim}} = 500$ time steps and the calibration length is $N_{\text{cal}} = 250$. For the model-based estimator validation, the transition-noise standard deviation is 10^{-4} and the Monte Carlo study uses 1000 independent calibration trials. Model-free regression sample counts and exploration parameters are algorithm-specific implementation settings and are stated in Chapter 5, while the plant, cost, offsets, closed-loop horizon, and execution cases remain common.

Table 3.5: Common numerical benchmark and experiment settings.

Setting	Value or interpretation
State dimension	$n = 3$.
Player input dimensions	$m_P = m_E = 1$.
Auxiliary engagement output	$C = [1 \ 0 \ -1]$.
State penalty	$Q = I_3$.
Input weights	$R_P = 1$ and $R_E = 1$.
True command offsets	$b_P = 0.040$ and $b_E = -0.010$.
Calibration length	$N_{\text{cal}} = 250$.
Closed-loop horizon	$N_{\text{sim}} = 500$ time steps.
Initial condition	$x_0 = [10 \ 5 \ -2]^T$.
Model-based calibration noise	Standard deviation 10^{-4} .
Monte Carlo estimator trials	1000.
Nominal interpretation	Reduced-order pursuit–evasion regulation benchmark; not a complete nonlinear vehicle model.

3.12. Assumptions and Limits of the Framework

Several delimitations follow directly from the equations in this chapter. First, the present plant is linear and time invariant. Nonlinear pursuit–evasion dynamics are outside the current derivation. Second, the full state is available to both policies. Output-feedback estimation appears in the broader source literature, but it is not added to the current four-algorithm comparison. Third, each player has one scalar input. Vector-valued acceleration and steering commands remain part of a future vehicle extension.

Fourth, the command offsets are constant over the calibration and simulation windows. Slowly varying biases may be approximated over short intervals, but abrupt faults or continuously drifting offsets would require online estimation, change detection, or adaptive compensation. Fifth, the compensation law assumes matched disturbances that enter through the player input channels. An unmatched disturbance cannot generally be canceled by subtracting a scalar command offset. Sixth, the model-free calibration assumes that the realized inputs can be measured. Without those measurements, additional observer or augmented-state structure would be required.

Finally, the current pursuit–evasion interpretation is local and reduced order. The thesis does not claim completed nonlinear vehicle simulation, QLab validation, or physical QCar experiments. Those application directions remain important because persistent acceleration and steering offsets

have clear physical meaning. However, establishing the disturbance-aware algorithmic foundation on the original linear benchmark is necessary before the framework is transferred to a higher-dimensional vehicle model.

These limits make the contribution more testable rather than less significant. The source algorithms are preserved, the disturbance model is explicit, the estimator assumptions are stated, and the numerical checks are reproducible. Consequently, later results can be traced to a specific equation, matrix property, or implementation decision rather than attributed to a broad and unverified notion of robustness.

3.13. Chapter Summary and Transition

This chapter developed the common mathematical framework used by the four disturbance-aware zero-sum algorithms. The nominal plant is a discrete-time linear system with one minimizing pursuer input and one maximizing evader input. A quadratic performance objective produces a feedback saddle solution described by the value matrix P , the action-value matrix H , the pursuer gain L , the evader gain K , and the GARE. The model-based methods evaluate these relationships using the known plant matrices, while the model-free methods estimate the corresponding quadratic action-value structure from sampled transitions.

The chapter then separated commanded and realized inputs through persistent offsets in both player channels. Those offsets introduce a constant affine term and may create a nonzero equilibrium even when the nominal feedback matrix remains Schur stable. Model-based transition residuals and model-free command-versus-measurement calibration provide two information-consistent estimation procedures. Subtracting the estimates from the policy commands leaves only the residual bias errors, while exact compensation recovers the nominal closed-loop dynamics.

The final portion of the chapter connected the mathematics to the common MATLAB evaluation. It defined the stability, curvature, conditioning, Riccati, Lyapunov, Bellman, rank, estimation, and compensation checks before introducing the numerical benchmark. The placement of the matrix values clarifies their purpose: they are the fixed reduced-order realization on which the common game, command offsets, calibration assumptions, and four execution cases are tested.

Chapter 4 develops the model-based GARE policy-iteration and value-iteration algorithms from this framework while preserving the nominal Riccati equations and adding transition-residual calibration and affine compensation. Chapter 5 develops the model-free quadratic Q-learning counterparts under the same plant, cost, player interpretation, and disturbance-aware protocol.

References Cited in Chapter 3

- [1] S. A. A. Rizvi and Z. Lin, *Output Feedback Reinforcement Learning Control for Linear Systems*. Cham, Switzerland: Birkhäuser, 2023.
- [2] A. Al-Tamimi, F. L. Lewis, and M. Abu-Khalaf, “Model-free Q-learning designs for linear discrete-time zero-sum games with application to H_∞ control,” *Automatica*, vol. 43, no. 3, pp. 473–481, 2007.
- [3] T. Başar and G. J. Olsder, *Dynamic Noncooperative Game Theory*, 2nd ed. Philadelphia, PA: SIAM, 1999.
- [4] M. Pachter and K. D. Pham, “Discrete-time linear-quadratic dynamic games,” *Journal of Optimization Theory and Applications*, vol. 146, no. 1, pp. 151–179, 2010.
- [5] I. E. Weintraub, M. Pachter, and E. Garcia, “An introduction to pursuit-evasion differential games,” in *Proceedings of the American Control Conference*, 2020.
- [6] A. Al-Tamimi, M. Abu-Khalaf, and F. L. Lewis, “Adaptive critic designs for discrete-time zero-sum games with application to H_∞ control,” *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, vol. 37, no. 1, pp. 240–247, 2007.
- [7] S. A. A. Rizvi and Z. Lin, “Output feedback Q-learning for discrete-time linear zero-sum games with application to H_∞ control,” *Automatica*, vol. 95, pp. 213–221, 2018.
- [8] K. R. Muske and T. A. Badgwell, “Disturbance modeling for offset-free linear model predictive control,” *Journal of Process Control*, vol. 12, no. 5, pp. 617–632, 2002.
- [9] G. Pannocchia and J. B. Rawlings, “Disturbance models for offset-free model-predictive control,” *AIChE Journal*, vol. 49, no. 2, pp. 426–437, 2003.
- [10] S. Gillijns and B. De Moor, “Unbiased minimum-variance input and state estimation for linear discrete-time systems,” *Automatica*, vol. 43, pp. 111–116, 2007.
- [11] R. A. Horn and C. R. Johnson, *Matrix Analysis*, 2nd ed. Cambridge, UK: Cambridge University Press, 2012.